

## COURSE OVERVIEW

1. **Introduction:** Before IEC 61131-3, every PLC manufacturer had its own language, its own IDE, and its own way of describing a contact or a coil. Moving between vendors meant starting over. The standard defined five languages, a common type system, and a module structure that works across implementations. This chapter explains what was unified and what was deliberately left vendor-specific.
2. **Data Types:** BOOL, INT, DINT, REAL, TIME, STRING, and user-defined structures are the building blocks of every IEC 61131-3 program. Type mismatches that cause silent data corruption look exactly like tuning problems or sensor faults at the process level. Getting types right comes first.
3. **Structured Text:** Pascal-style text with IF/THEN, FOR, WHILE, CASE, and function calls. The most expressive of the five languages and the one that feels most familiar to software engineers. Also the one most PLC programmers reach for last, which is a mistake this chapter corrects.
4. **Ladder Diagram:** Relay logic drawn as rungs on a rail. Every PLC vendor supports it. Electricians who have never written software can read it. Understanding Ladder is non-optional for anyone who will touch existing automation code, because most existing automation code is in Ladder.
5. **Function Block Diagram:** Data flows left to right between connected function blocks. No explicit control flow. The natural language for process control engineers working with PID loops, filters, and signal conditioning chains where the data path matters more than execution order.
6. **Sequential Function Chart:** Steps and transitions model a process that moves through discrete states. A batch filling sequence, a motor start-stop sequence, a conveyor interlocking sequence — any process with a defined beginning, middle, and end is cleaner in SFC than in Ladder with timer flags.
7. **Program Organization Units:** Functions, Function Blocks, Programs, and Configurations are the module system. A well-defined Function Block can be reused across vendors without modification. Building libraries that survive hardware changes is the point of this chapter.
8. **RTAC Integration:** The SEL RTAC runs IEC 61131-3 programs through ACSELERATOR Architect. The same language that controls a tank level also controls a transformer bay. The chapter covers how PLC programming concepts translate to substation automation.
9. **Troubleshooting:** Implicit type conversion that changes values without warning. Scan cycle overruns that make outputs appear delayed. Latching coils that never reset. Runtime faults that stop the PLC silently. Each failure has a diagnosis procedure.
10. **Practice Lab:** A temperature PID controller in Structured Text with manual/auto switching. A motor starter in Ladder with overload and remote stop. A batch fill sequence in SFC with a weight setpoint. A reusable analog scaling Function Block with engineering unit conversion.

### Certification Relevance

- **PLCopen Certified** — Certification exams test IEC 61131-3 language knowledge, POU's, and programming best practices directly.
- **ISA CCST** — PLC programming, ladder logic, and control system configuration are core tested topics at all levels.
- **Vendor Credentials** — Siemens TIA Portal, Rockwell Studio 5000, and Beckhoff TwinCAT all use IEC 61131-3 as their programming foundation.

### Next Steps

1. Take IEC 61131-3 before the RTAC guide. The logic concepts transfer directly to ACSELERATOR Architect.
2. Download CODESYS (free, IEC 61131-3 compliant IDE) and run the lab exercises in a soft PLC.
3. Review the PLCopen Function Block libraries for motion, communications, and safety.
4. PLCopen certification or a vendor credential (TIA Portal, Studio 5000) is the right first PLC-specific credential.